

CampusEats — Partner SOAP Integration

Group Name 24	
Paras Pandey	20251651068
Pankaj Singh	20251651067
Pranav Bhavsar	20251651069
Jayesh Badole	20251651049
Shivam Phogat	20251651082

Partner: SecurePay — an external payment gateway. **Operation integrated:** `processPayment`, invoked by CampusEats when a student confirms an order in the `placeOrder` flow.

The rest of CampusEats (menus, orders, ratings, order-status polling) is REST because those resources are simple, cacheable, and change-tolerant — a new field can be added to a JSON response without breaking clients. The payment edge is different: it is a financial transaction that must be described by a strict, machine-checkable contract (WSDL/XSD) so card data, amounts and currencies cannot silently drift between CampusEats and the bank-facing gateway. It also needs message-level security — a WS-Security UsernameToken travelling inside the SOAP Header itself rather than relying only on transport-level TLS, since the message may be logged, queued or relayed by intermediaries between CampusEats and SecurePay. Finally, a payment is a transactional operation with well-defined success/failure semantics that SOAP's formal fault model (`errorCode` / `errorMessage`) supports more rigorously than a plain REST error body.

1. Partner WSDL

The full contract is submitted separately as `partner.wsdl`. It defines all six required elements for the `processPayment` operation:

- **types** — XSD schema (targetNamespace `http://securepay.com/payment/types/`) for `processPaymentRequest`, `processPaymentResponse` and `paymentFault`
- **message** — `processPaymentRequestMessage`, `processPaymentResponseMessage`, `paymentFaultMessage`
- **portType** — `PaymentPortType` with operation `processPayment` (input / output / fault)
- **binding** — `PaymentSoapBinding`, SOAP over HTTP, document/literal style
- **service / port** — `SecurePayService` / `PaymentPort` with a live-looking endpoint

Endpoint (service/port): `https://api.securepay.com/soap/payment`

SOAPAction (from binding): `http://securepay.com/payment/processPayment`

2. SOAP Messages

Three hand-written envelopes are submitted as separate files, each matching the WSDL element-for-element: `soap-request.xml` (a WS-Security UsernameToken credential block in the Header, `processPaymentRequest` in the Body), `soap-response.xml` (`processPaymentResponse`, status `SUCCESS`), and `soap-fault.xml` (a `soap:Fault` whose detail carries a `paymentFault` with `errorCode` `SP-402`, “Card Declined: Insufficient Funds”).

3. HTTP Binding

This is the literal HTTP POST that carries `soap-request.xml` to the endpoint declared in the WSDL's `service/port`, using the `SOAPAction` declared in the binding:

```
POST /soap/payment HTTP/1.1
Host: api.securepay.com
Content-Type: text/xml; charset=utf-8
Content-Length: 943
SOAPAction: "http://securepay.com/payment/processPayment"

<?xml version="1.0" encoding="utf-8"?> <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:tns="http://securepay.com/payment/types/"> <soapenv:Header> <wsse:Security
xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd"
soapenv:mustUnderstand="1"> <wsse:UsernameToken>
    <wsse:Username>campuseats_partner_24</wsse:Username>
    <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-
1.0
    </wsse:UsernameToken>
</wsse:Security>
</soapenv:Header>
<soapenv:Body>
    <tns:processPaymentRequest>
        <tns:cardHolderName>Paras Pandey</tns:cardHolderName>
        <tns:cardNumber>9258647312583642</tns:cardNumber>
        <tns:cvv>887</tns:cvv>
        <tns:expiryDate>12/28</tns:expiryDate>
        <tns:amount>450.50</tns:amount>
        <tns:currency>INR</tns:currency>
    </tns:processPaymentRequest>
</soapenv:Body>
</soapenv:Envelope>
```

4. Discovery (modern form)

CampusEats obtains the contract in one of two ordinary ways: (a) a **direct URL** the gateway publishes next to its docs, `https://api.securepay.com/soap/payment?wsdl`, fetched once at integration time and vendored into the CampusEats payment-adapter repo; or (b) a **lookup** in the organisation's internal API catalogue (an internal developer portal / service registry) where partner contracts are indexed by business name and capability, so a new developer searches “payment” instead of being emailed a link. No UDDI server is run; the catalogue entry below is a single JSON record playing the same role a UDDI `tModel/businessService` pair used to play.

Catalogue / registry entry:

```
{
  "businessEntity": {
    "name": "SecurePay",
    "description": "Third-party card payment gateway"
```

```

},
"businessService": {
  "name": "SecurePayService",
  "operation": "processPayment"
},
"bindingTemplate": {
  "accessPoint": "https://api.securepay.com/soap/payment",
  "soapAction": "http://securepay.com/payment/processPayment"
},
"tModel": {
  "name": "PaymentSoapBinding-v1",
  "overviewDoc": "https://api.securepay.com/soap/payment?wsdl"
}
}

```

5. Fault Mapping

SecurePay's SOAP fault is caught by the CampusEats payment adapter and translated into the error the Assignment 2 `placeOrder` contract already promised. The partner's internal vocabulary (`errorCode`, `faultstring` wording) never reaches the student-facing API.

SecurePay (partner) fault	CampusEats <code>placeOrder</code> error
<code>errorCode</code> = SP-402, <code>faultcode</code> = <code>soapenv:Client</code> ("Card Declined: Insufficient Funds")	402 <code>PAYMENT_DECLINED</code> — "Your payment was declined. Try another card."
other SP-4xx <code>errorCode</code> values (e.g. expired card, invalid CVV)	402 <code>PAYMENT_DECLINED</code> — same generic student-facing message
<code>soapenv:Server</code> fault / connection timeout	503 <code>PAYMENT_UNAVAILABLE</code> — "Payment service is temporarily unavailable, please retry."

The adapter's fault handler pattern-matches on SecurePay's `errorCode`/`faultcode` and rethrows one of CampusEats's own, already-documented error codes; it never surfaces SecurePay's field names, codes, or raw `faultstring` text to the client.